CS1004 – Object Oriented Programming

Lecture # 16
Wednesday, April 14, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Operator Overloading

Operator Overloading

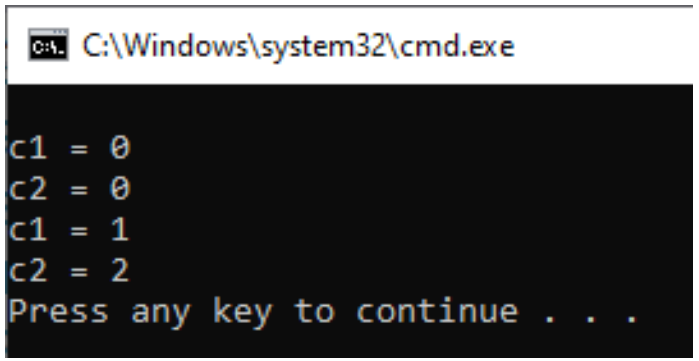
- One of the most exciting feature of OOP
- Transforms complex function calls to obvious ones
- Example
  - `d3.addobjects(d1, d2);`

    OR
  - `d3 = d1.addobjects(d2);`
  - Converted to a much more readable
  - `d3 = d1 + d2;`
- Gives normal C++ operators +, -, ++, <= and >= additional meaning when applied to user-defined data types

Overloading Unary Operator pre-increment (++)

```
class Counter
{
private:
unsigned int count; //count
public:
Counter() : count(0){} //constructor
unsigned int get_count(){ //return count
            return count;
}
void operator ++(){//increment (prefix)
            ++count;
}
};
```

```
int main()
{
Counter c1, c2; //define and initialize
cout << "\nc1 = " << c1.get_count();     cout <<
"\nc2 = " << c2.get_count();
++c1; //increment c1
++c2; //increment c2
++c2; //increment c3
cout << "\nc1 = " << c1.get_count();     cout <<
"\nc2 = " << c2.get_count()
            << endl;
return 0;
}
```



Overloading Unary Operator pre-increment (++)

- The code in the last slide does not allow the following statement to execute
- `c1 = ++c2;`
- Because the return type is `void`
- To overcome this problem

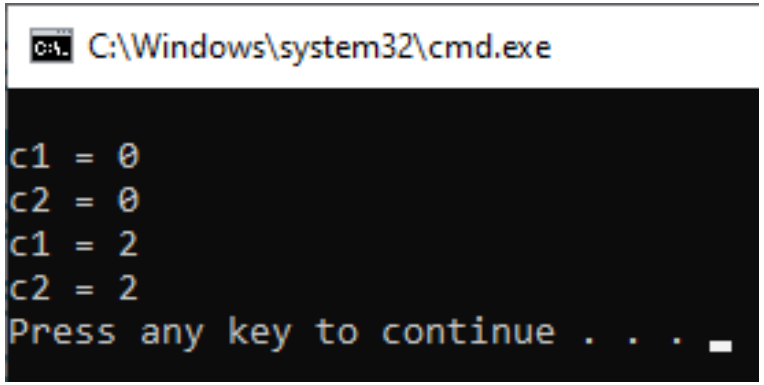
```
Counter operator ++ ()
{
++count; //increment count
Counter temp; //make a temporary
                                    //Counter
temp.count = count; //give it same
                                    //value as this obj
return temp; //return the copy
}
```

```
int main()
{
Counter c1, c2;
cout << "\nc1 = " << c1.get_count();
cout << "\nc2 = " << c2.get_count();

++c1; //increment c1

c2 = ++c1;

cout << "\nc1 = " << c1.get_count();
cout << "\nc2 = " << c2.get_count()
        << endl;
return 0;
}
```



Nameless Temporary Objects

```
Counter operator ++ ()
{
++count; //increment count
Counter temp; //make a temporary
                        //Counter
temp.count = count; //give it same
                        //value as this
obj
return temp; //return the copy
}
```

- Can be made more convenient to return temporary objects from functions and overloaded operators

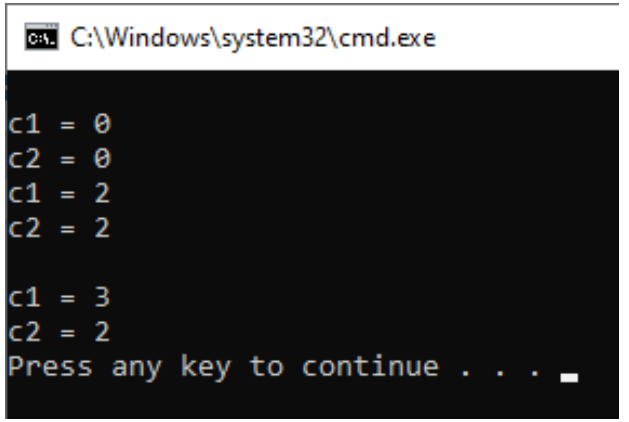
- By adding overloaded constructor and new definition of ++

```
Counter(int c):count(c) //constructor
{ }
Counter operator ++() //increment count
{
++count; // increment count
//return an unnamed temporary
//object
return Counter(count);
}
```

Post-increment Counter++

```
//increment count (postfix)
Counter operator ++ (int)
{
//return an unnamed temporary
return Counter(count++);
//object initialized to this
//count, then increment count
}
```

```
int main()
{
Counter c1, c2;
cout << "\nc1 = " << c1.get_count();    cout << "\nc2 = " <<
c2.get_count();
++c1; //increment c1
c2 = ++c1;
cout << "\nc1 = " << c1.get_count();
cout << "\nc2 = " << c2.get_count()
             << endl;
c2 = c1++;
cout << "\nc1 = " << c1.get_count();
cout << "\nc2 = " << c2.get_count()
             << endl;
return 0;
}
```



Pre and post increment

- Pre-increment
  - `Counter operator ++ () //increment count (prefix)`
- Post-increment
  - `Counter operator ++ (int) //increment count (postfix)`
- The only different is int in the parenthesis
  - It is not actually a parameter
  - Just a signal to compiler to create post-increment

Pre and post decrement

- Pre-increment
  - `Counter operator -- () //decrement count (prefix)`
- Post-increment
  - `Counter operator -- (int) //decrement count (postfix)`

Overloading Binary Operators

Arithmetic Operators

Performing sum of two objects

```
class Distance //English Distance class
{
private:
int feet;
float inches;
public: //constructor (no args)
Distance() : feet(0), inches(0.0) { }
//constructor (two args)
Distance(int ft, float in) : feet(ft),

inches(in) { }
void showdist() const //display distance
{
                        cout << feet << "' - "
                                                << inches << '"';
}
void getdist() //get length from user
{
                        cout << "\nEnter feet : "; cin >> feet;
                        cout << "Enter inches : "; cin >> inches;
}
Distance sum(Distance d1, Distance d2);
};
```

```
void Distance::sum(Distance d1, Distance d2){
feet = d1.feet + d2.feet;
inches = d1.inches + d2.inches;
if (inches >= 12.0) //if total exceeds 12.0,
{ //then decrease inches
            inches -= 12.0; //by 12.0 and
            feet++; //increase feet by 1
}
}
int main()
{
Distance dist1, dist2, dist3;
dist1.getdist();
Distance dist2(11, 6.25);
dist3.sum(dist1, dist2);
dist3 = dist1 + dist2; //single '+' operator
cout << "dist1 = "; dist1.showdist(); cout << endl;
cout << "dist2 = "; dist2.showdist(); cout << endl;
cout << "dist3 = "; dist3.showdist(); cout << endl;
return 0;
}
```

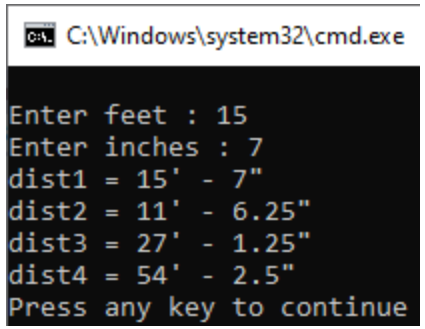
```
C:\Windows\system32\cmd.exe

Enter feet : 15
Enter inches : 7
dist1 = 15' - 7"
dist2 = 11' - 6.25"
dist3 = 27' - 1.25"
Press any key to continue .
```

Performing sum of two objects using Overloaded +

```
class Distance //English Distance class
{
private:
int feet;
float inches;
public:
Distance() : feet(0), inches(0.0){ }
Distance(int ft, float in) : feet(ft), inches(in){ }
void getdist() //get length from user
{
            cout << "\nEnter feet : "; cin >> feet;
            cout << "Enter inches : "; cin >> inches;
}
void showdist() const //display distance
{
            cout << feet << "' - " << inches << '"';
}
Distance operator + (Distance) const; //add 2 distances
};
```

```
Distance Distance::operator + (Distance d2) const //return sum
{
int f = feet + d2.feet; //add the feet
float i = inches + d2.inches; //add the inches
if (i >= 12.0) //if total exceeds 12.0,
{ //then decrease inches
            i -= 12.0; //by 12.0 and
            f++; //increase feet by 1
} //return a temporary Distance
return Distance(f, i); //initialized to sum
}
int main()
{
Distance dist1, dist3, dist4; //define distances
dist1.getdist(); //get dist1 from user
Distance dist2(11, 6.25); //define, initialize dist2
dist3 = dist1 + dist2; //single '+' operator
dist4 = dist1 + dist2 + dist3; //multiple '+' operators
cout << "dist1 = "; dist1.showdist(); cout << endl;
cout << "dist2 = "; dist2.showdist(); cout << endl;
cout << "dist3 = "; dist3.showdist(); cout << endl;
cout << "dist4 = "; dist4.showdist(); cout << endl;
return 0;
}
```



Redefining + as concatenation in Strings

```
class String{
enum {SZ=80}; //static const int SZ = 80;
char str[SZ];
public:
String() { strcpy(str, ""); }
String(char st[]) { strcpy(str, st); }
void display() const{ cout << str; }
String operator +(String st) const {
                    String temp;
                    if (strlen(str) + strlen(st.str) < SZ)
                    {
                                        strcpy(temp.str, str);
                                        strcat(temp.str, st.str);
                    }
                    else
                    {
                                        cout << "\nString overflow";
                                        exit(1);
                    }
return temp;
}
};
```

```
int main()
{
String s1 = "\nHello";
String s2 = " World!";
String s3;
s1.display();
s2.display();
s3.display();
s3 = s1 + s2; //add s2 to s1,
        //assign to s3
s3.display(); //display s3
cout << en
return 0;
}
```

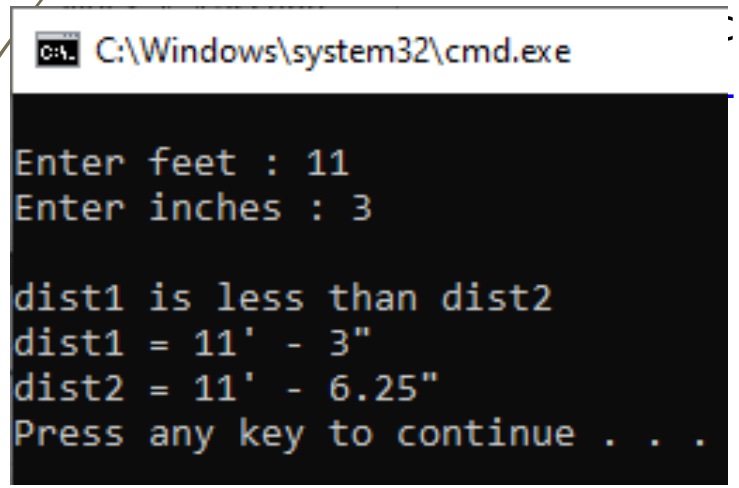
```
C:\Windows\system32\cmd.exe

Hello World!
Hello World!
Press any key to continue . . .
```

Comparison operators <

- Adding a new < overloaded function to the Distance class

```
bool Distance::operator < (Distance
dt) const
{
float d1 = feet + inches / 12;
                          hes / 12;
                          se;
```




```
int main()
{
Distance dist1, dist3, dist4;
dist1.getdist();
Distance dist2(11, 6.25);
if (dist1 < dist2)
            cout << "\ndist1 is less than"
<< "dist2";
else
            cout << "\ndist1 is greater than"
                        << " (or equal to) dist2";
cout << endl;
cout << "dist1 = ";
dist1.showdist();
cout << endl;
cout << "dist2 = ";
dist2.showdist();
cout << endl;
}
```

Arithmetic assignment Operator +=

- To implement += we have two options

```
void Distance::operator +=(Distance d)
{
feet += d.feet;
inches += d.inches;
if (inches > 12)
{
        feet++;
        inches -= 12;
}
}
```

- We will be able to execute

```
dist2 += dist1;
```

- But the instruction below will be a syntax error

```
(dist3 += dist1) += dist2;
```

- To overcome this issue we have to introduce a different += overloaded function

```
Distance& Distance::operator +=(Distance d)
{
feet += d.feet;
inches += d.inches;
if (inches > 12)
{
            feet++;
            inches -= 12;
}
return *this;
}
```

- Now we'll be able to execute

```
dist2 += dist1;
```

- As well as

```
(dist3 += dist1) += dist2;
```

Overloading [] operator

- To implement a safe array that takes care of array out of bound problem we can implement overloaded [] operator

```
const int LIMIT = 100;

class safearay
{
private:
int arr[LIMIT];
public:
int& operator[](int n)
{
                if (n< 0 || n >= LIMIT)
                {
                                cout << "\nIndex out of bounds";
                                exit(1);
                }
                return arr[n];
}
};
```

```
int main()
{
safearay arr;
for (int i = 0; i < LIMIT; ++i)
                arr[i] = i * 10;

for (int i = 0; i < LIMIT; ++i)
                cout << "Element " << i
                                << " = " << arr[i] << endl;
return 0;
}
```

- We need to return the reference because array can be used at the left side of assignment operator as well

Notes about operator Overloading

- An overloaded operator always requires one less argument than its number of operands
  - one operand is the object of which the operator is a member

Questions